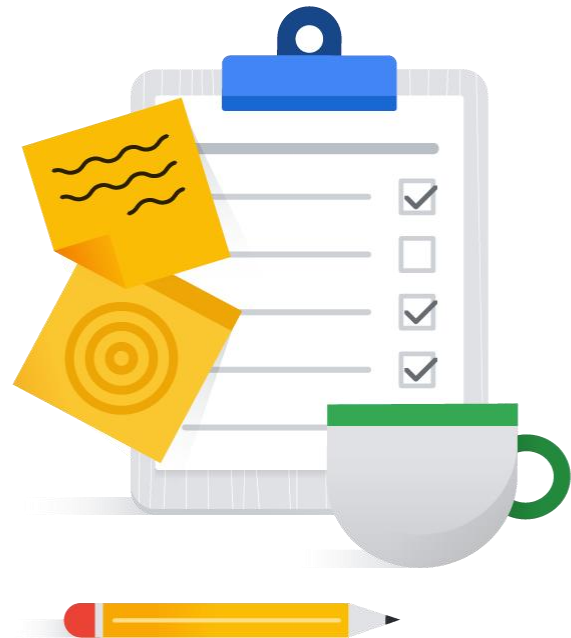


Security in BigQuery

09

Topics

01	Secure Resources with IAM
02	Access Control with Authorized Views
03	Secure Data with Classification
04	Encryption
05	Data Discovery and Governance



Security, encryption, and auditing for BigQuery

Key Management

Customer-managed
encryption keys



Cloud Audit Logs

Admin activity
System event
Data access

Authentication

Service accounts
User accounts



Access Control

Cloud IAM roles
Permissions (custom)
Project, Dataset,
Table, View, Row,
Column

Principle of least privilege

- Users should only be able to do the tasks that are required by their jobs.
- This should also apply to machine instances and run-time processes.
- Use IAM to enforce this principle.
- Identify users with their login.
- Identify machines and code using service accounts.
- Assign IAM roles to users and service accounts to restrict what they can do.

Separation of duties

- No one person can change or delete data without being detected.
- No one person can steal sensitive data.
- No one person is in charge of designing, implementing, and reporting on sensitive systems.

For example, the people who write the code shouldn't deploy the code, and those who deploy the code shouldn't be able to change it.

- Use multiple projects to separate duties.
- Different people can be given different rights in different projects.
- Use folders to help organize projects.

01



**Secure resources
with IAM**

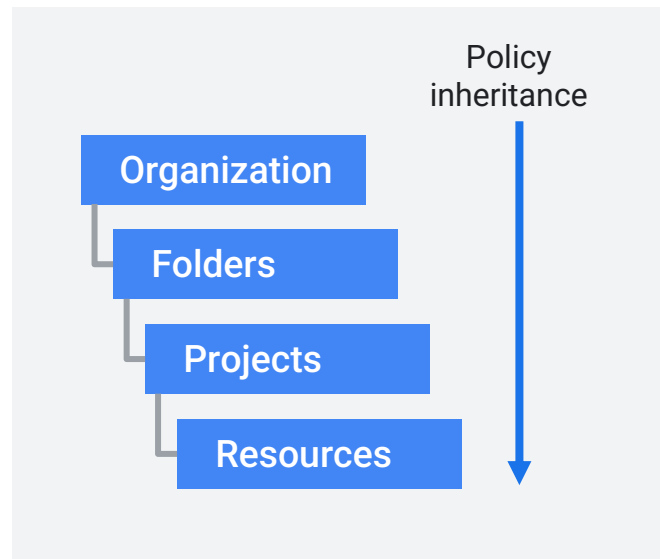
Use Identity and Access Management policies to secure environments and manage resources

Grant roles to Google groups rather than individuals

- Groups can be more granular than job roles.
- Use multiple groups for better control (such as *view only*).

Roles

- Prefer pre-defined roles over custom roles.
- Grant roles at the smallest scope needed (least privilege).
- Limit use of "owner" and "editor" roles.
- Consider hierarchy inheritance when assigning roles.



IAM and BigQuery: Identities

Specific end user: **example@mydomain.com**

User-created service account: **my-custom-service@myproject.iam.gserviceaccount.com**

Per-product, Per-product service account: **923657641019-compute@developer.gserviceaccount.com**

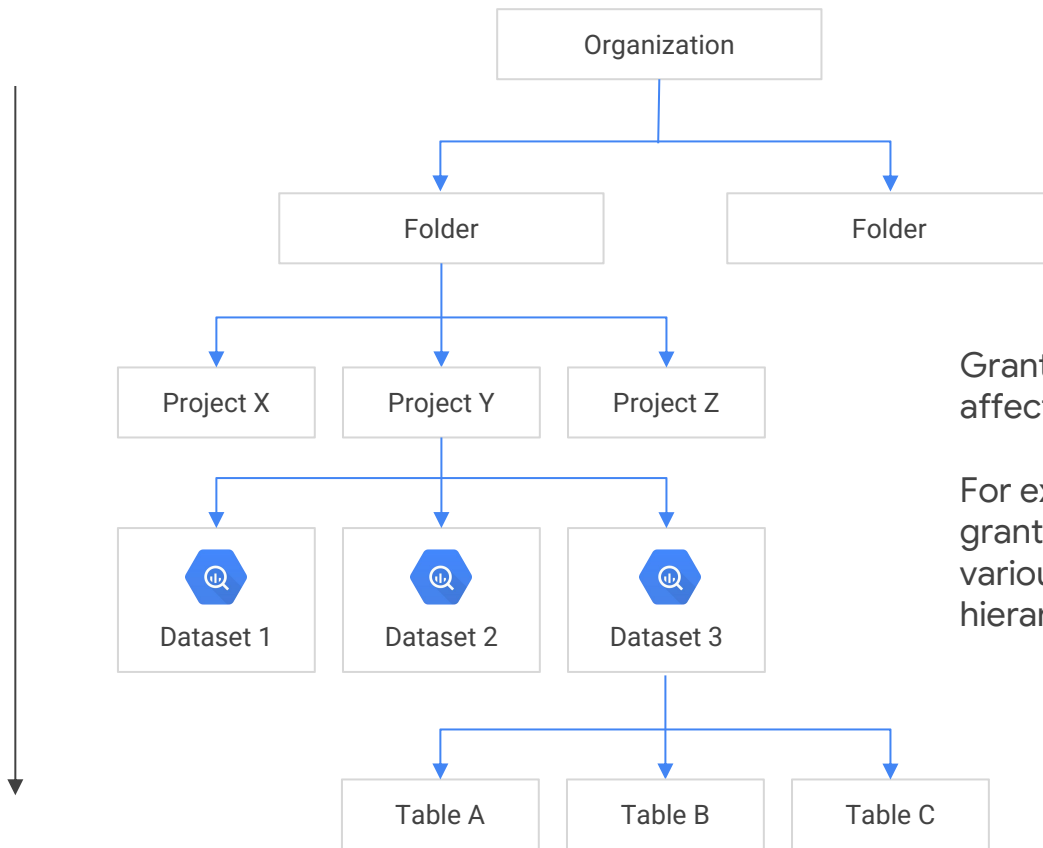
Group: **data-analysts@mydomain.com**

Domain: **mydomain.com**

Special identifiers: **allUsers** or **allAuthenticatedUsers**

Access control

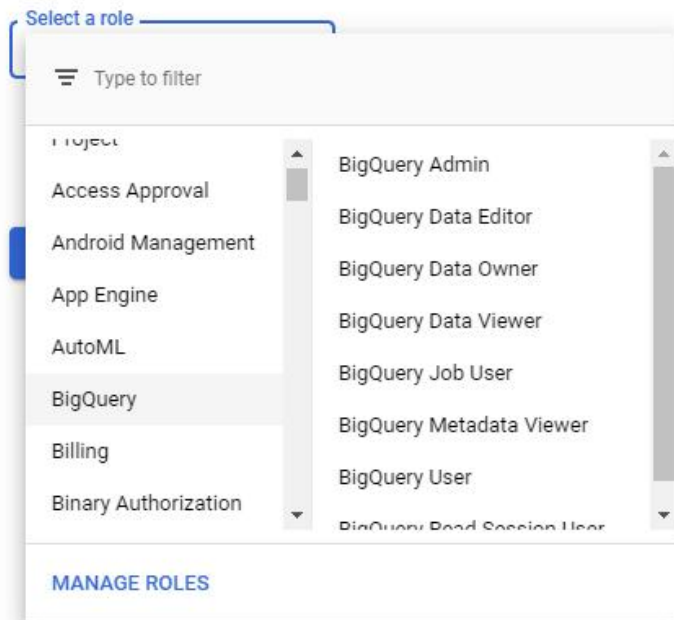
Permission inheritance



Granting a permission to a resource affects **all resources below it**.

For example, consider the effect of granting **bigquery.tables.delete** at various places in this resource hierarchy.

BigQuery provides predefined roles for controlling access to resources



Grants

IAM grants permission to perform specific actions.

Access control refers to datasets, tables, views, or columns.

Use [authorized views](#) to restrict at a finer-grained level.

You can assign the following BigQuery predefined IAM roles

Admin roles

BigQuery Admin
(roles/bigquery.admin)

BigQuery Connection Admin
(roles/bigquery.connectionAdmin)

BigQuery Resource Admin
(roles/bigquery.resourceAdmin)

Data roles

BigQuery Data Editor
(roles/bigquery.dataEditor)

BigQuery Data Owner
(roles/bigquery.dataOwner)

BigQuery Data Viewer
(roles/bigquery.dataViewer)

Viewer roles

BigQuery Filtered Data Viewer
(roles/bigquery.filteredDataViewer)

BigQuery Resource Viewer
(roles/bigquery.resourceViewer)

BigQuery Metadata Viewer
(roles/bigquery.metadataViewer)

Other roles

BigQuery Read Session User
(roles/bigquery.readSessionUser)

BigQuery Connection User
(roles/bigquery.connectionUser)

BigQuery Resource Editor
(roles/bigquery.resourceEditor)

BigQuery Job User
(roles/bigquery.jobUser)

BigQuery User
(roles/bigquery.user)

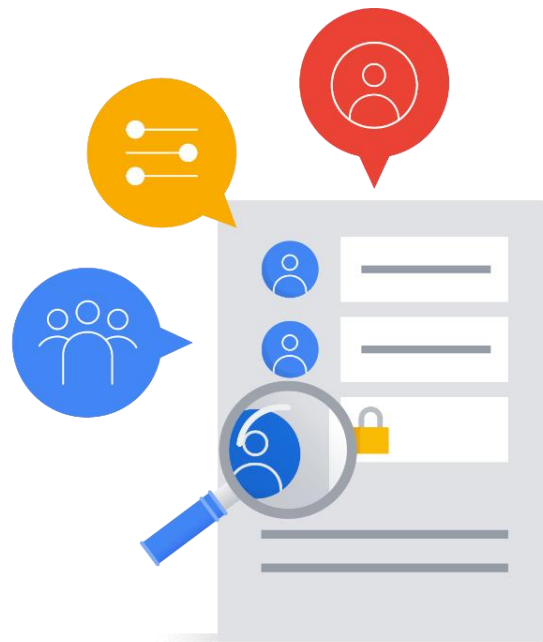
Masked Reader
(roles/bigquerydatapolicy.maskedReader)

BigQuery IAM roles

Role	Description
BigQuery Admin	Can do everything in BigQuery: create and read data, run jobs, set IAM policies, etc.
BigQuery Data Owner	Has read/write access to data, plus can grant access to other users and groups by setting IAM policies.
BigQuery Data Editor	Has read/write access to data.
BigQuery Data Viewer	Has read-only access to data.
BigQuery Job User	Can create and run jobs, but has no access to data.
BigQuery User	Can run jobs, create datasets, list tables, and save queries, but has no default access to data.

Using BigQuery IAM roles

- Assign groups (or users) to BigQuery IAM roles.
 - Managing users in groups is generally considered a best practice.
- Provide groups or users with access to datasets.
 - Can be viewer, editor, or owner.
- The user who created a dataset is the owner.
 - Can add additional users and other owners.



IAM Project Roles and inherited BigQuery permissions

When a project is created, BigQuery grants the **Owner** role to the user who created the project.

Basic role	Capabilities
Viewer	● Can start a job in the project. Additional dataset roles are required depending on the job type. ● Can list and get all jobs, and update jobs that they started for the project. ● If you create a dataset in a project that contains any viewers, BigQuery grants those users the bigquery.dataViewer predefined role for the new dataset.
Editor	● Same as Viewer, plus: ○ Can create a new dataset in the project. ○ If you create a dataset in a project that contains any editors, BigQuery grants those users the bigquery.dataEditor predefined role for the new dataset.
Owner	● Same as Editor, plus: ○ Can list all datasets in the project. ○ Can delete any dataset in the project. ○ Can list and get all jobs run on the project, including jobs run by other project users. ○ If you create a dataset, BigQuery grants all project owners the bigquery.dataOwner predefined role for the new dataset.



Access Control with Authorized Views

Authorized views

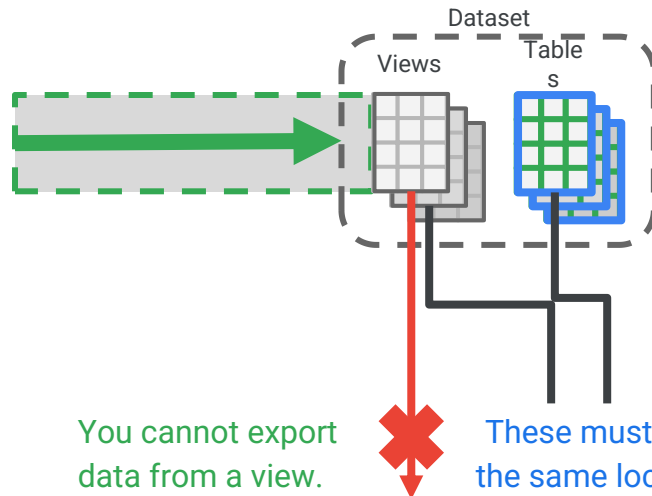
What if I want admins or super users to see all the data in a table and want others to see only a subset of the data?

- Use views to provide row or column level permissions to datasets.
- Create a second dataset with different permissions from the first.
- Add a view to the second dataset that selects the subset of data you want to expose from the first dataset.
- In the first dataset, you have to give access to the view in the second dataset (known as the *authorized view*) so it can access the underlying data.

Views add another degree of access control

A **view** is a virtual table defined by a SQL query.

An authorized view allows you to share data externally without sharing the underlying table.



```
CREATE VIEW dsB.myview AS  
SELECT  
  name, number  
FROM dsB.mytable  
WHERE year > 1950
```

You cannot export data from a view.

These must be in the same location.

Design pattern: Authorized views

Authorized views permit users to query a view without having read access to the underlying tables.

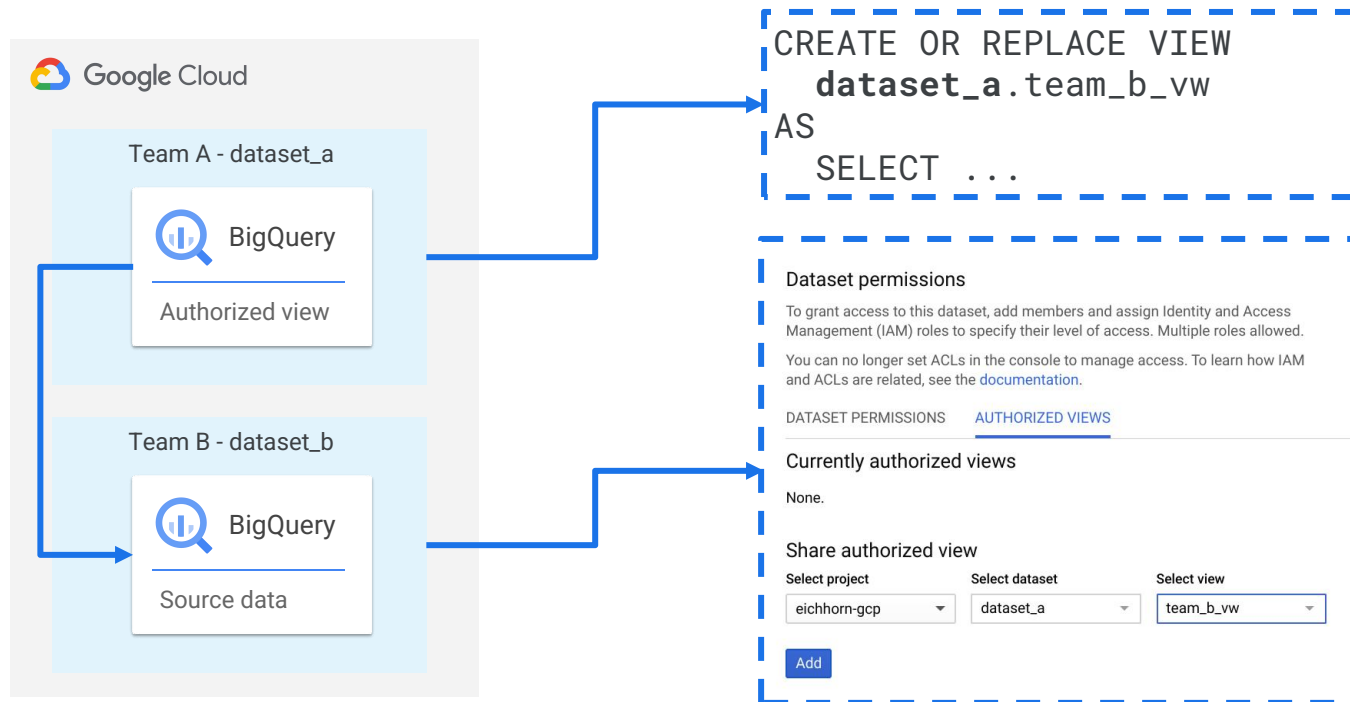
Create logical views into your dataset that filter based on user roles for more **fine-grained** access control.

Authorized views must be placed in a dataset separate from the restricted dataset.

Use `SESSION_USER()` to add your own custom authorization logic; for example, JOIN against user lookup table to manage access by user.

```
#standardSQL
SELECT column1, column2
FROM `dataset.table`
WHERE allowed_viewer = SESSION_USER()
```

Creating an authorized view



Take advantage of authorized views and SESSION_USER to achieve row-level security

```
#standardSQL/  
SELECT customer, id  
FROM private.customers  
WHERE allowed_viewer = SESSION_USER()
```

Add another column to your table for the user who is allowed to see the row

```
{engineers, example@mydomain.com}  
{engineers, some_engineer@google.com}  
{administrators, some_admin@google.com}  
{sales, some_salesperson@google.com}  
...
```

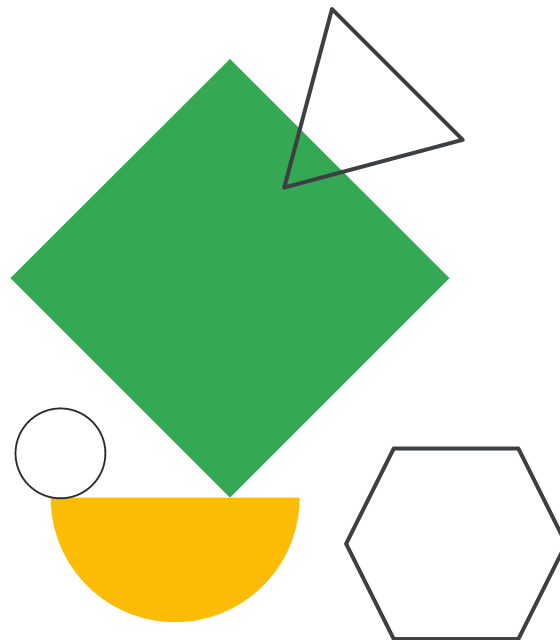
Make the allowed_viewer column a repeated field.

```
#standardSQL/  
SELECT c.customer, c.id  
FROM private.customers c  
INNER JOIN (  
    SELECT group  
    FROM  
private.access_control  
    WHERE user_name = SESSION_USER()) g  
ON c.allowed_group = g.group
```

Manage the access control group mapping (private.access_control) separately from the data table (private.customers).

Demo

Protecting data with a view and
SESSION_USER





03

Secure Data with Classification

Fined-grained access with column-level and row-level security

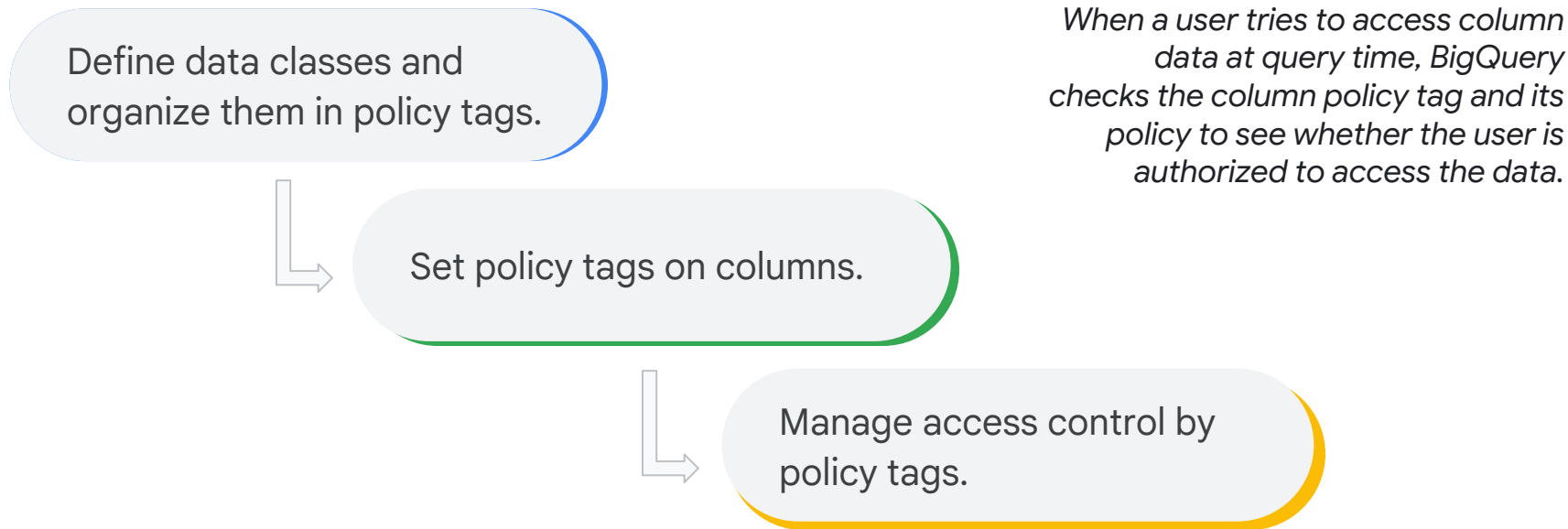


Control who can view data in select columns in BigQuery by creating policy tags in Data Catalog.

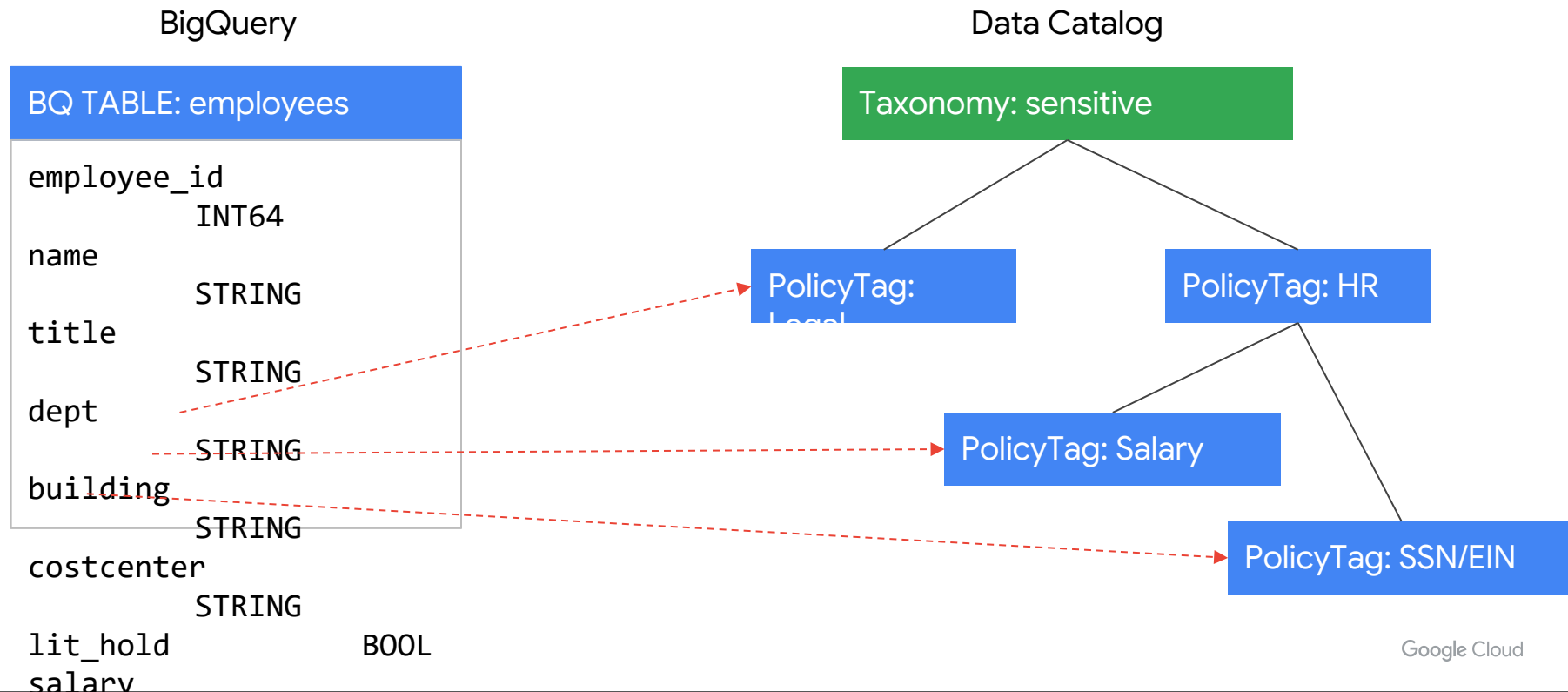


Filter data and enable access to specific rows in a table based on qualifying user conditions.

Column-level access control workflow



Define taxonomy and policy tags



Set policy tags on columns

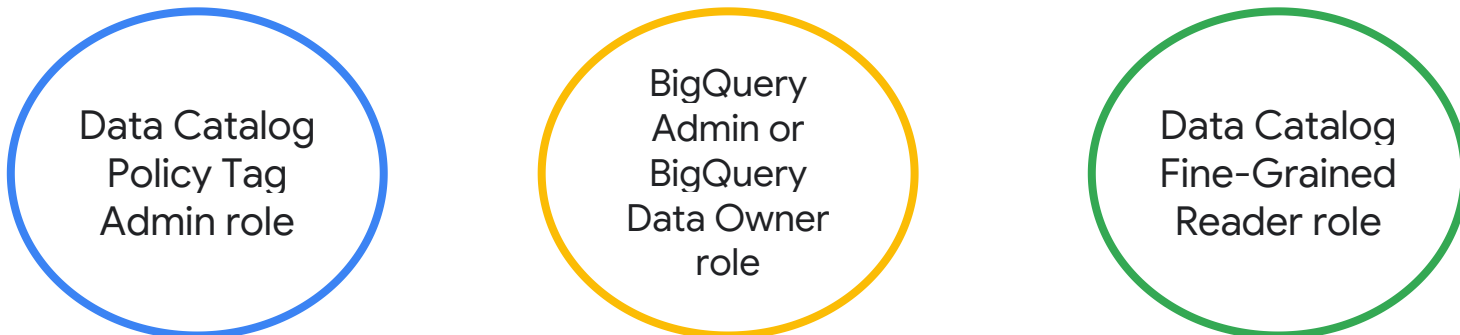
Modify *schema.json* to use policy tag:

```
[  
  ...  
  {  
    "name": "ein",  
    "type": "STRING",  
    "mode": "REQUIRED",  
    "policyTags": {  
      "names": ["projects/project-id/locations/location/taxonomies/taxonomy-id/policyTags/policytag-id"]  
    }  
  },  
  ...  
]
```

Update the schema:

```
bq update project-id:dataset.table schema.json
```

Roles used with column-level access control



Data Catalog
Policy Tag
Admin role

- Create, read, update, and delete taxonomies and policy tags.
- Get and set IAM policies on policy tags.

BigQuery
Admin or
BigQuery
Data Owner
role

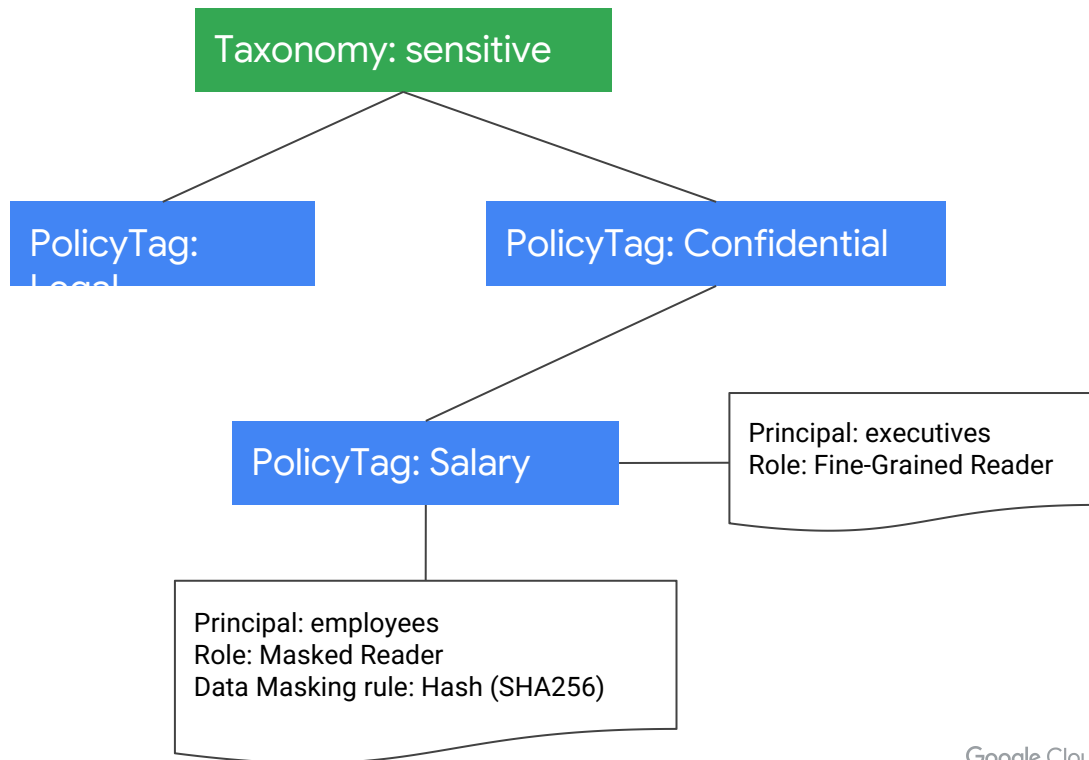
- Create, read, update, and delete data policies.
- Get and set IAM policies on data policies.

Data Catalog
Fine-Grained
Reader role

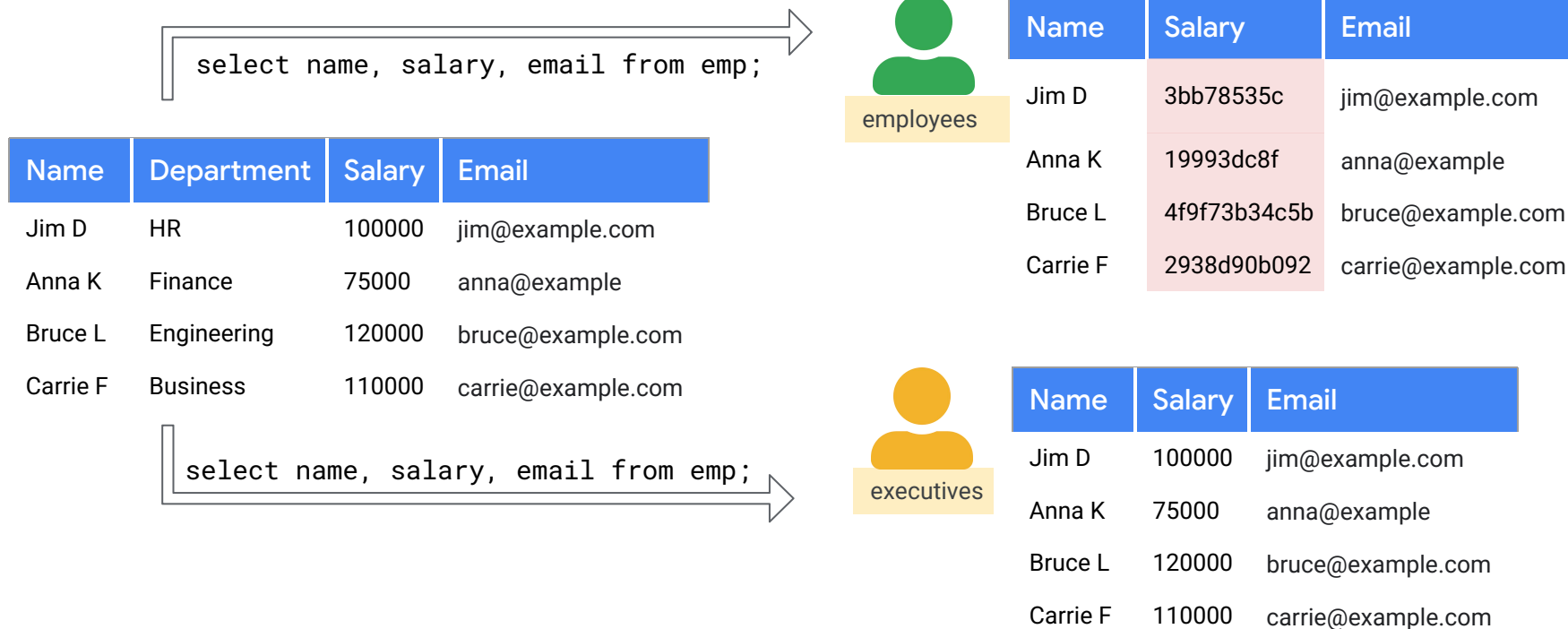
- Can access the content of columns restricted by a policy tags.

Dynamic data masking

- Mask sensitive columns so that tables can be shared with larger groups.
- Modifying existing queries by excluding the columns that the user cannot access is not necessary.
- Write a data policy once, associate it with a policy tag, and apply the policy tag to any number of columns.



Different levels of visibility with dynamic data masking



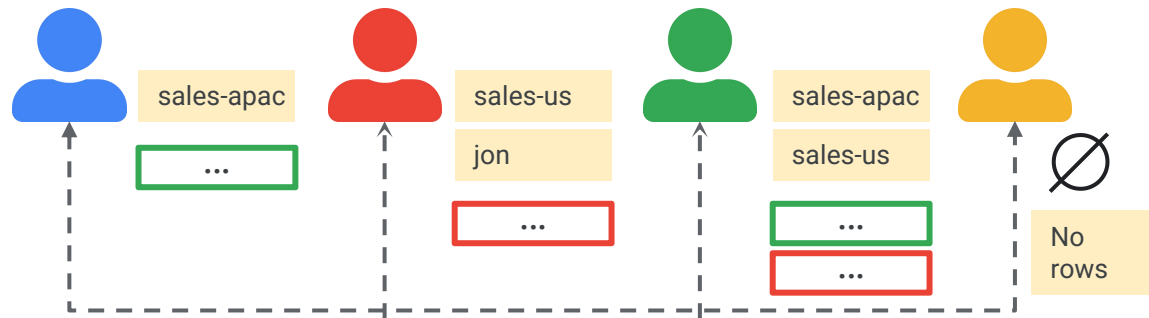
Row-level security in BigQuery

WHAT is it?	Involves the creation of row-level access policies on a target BigQuery table
HOW does it work?	Filters to hide or display certain rows of data, depending on whether a user or group is in an allowed list
WHO can create?	An authorized user (with the BigQuery Admin or BigQuery DataOwner IAM roles) can create row-level access policies on a BigQuery table.

Filter row data based on region

```
CREATE ROW ACCESS POLICY
  apac_filter
ON
  dataset1.table1
GRANT TO
  ("group:sales-apac@example.com")
FILTER USING
  (Region="APAC");
```

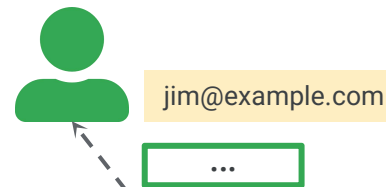
```
CREATE ROW ACCESS POLICY
  us_filter
ON
  dataset1.table1
GRANT TO
  ("group:sales-us@example.com",
   "user:jon@example.com")
FILTER USING
  (Region="US");
```



Partner	Contact	Country	Region
Example Customers Corp	alice@examplecustomers.com	Japan	APAC
Example Enterprise Group	bob@exampleenterprisegroup.com	Singapore	APAC
Example HighTouch Co.	carrie@examplehightouch.com	USA	US
Example Buyers Inc.	david@examplebuyersinc.com	USA	US

Filter row data based on sensitive data

```
CREATE ROW ACCESS POLICY
    salary_personal
ON
    dataset1.table1
GRANT TO
    ("domain:example.com")
FILTER USING
    (Email=SESSION_USER
    ());
```



Name	Department	Salary	Email
Jim D	HR	100000	jim@example.com
Anna K	Finance	75000	anna@example.com
Bruce L	Engineering	120000	bruce@example.com
Carrie F	Business	110000	carrie@example.com

Other examples of row access policies

Create a row access policy with ***allAuthenticatedUsers*** as the grantees:

```
CREATE ROW ACCESS POLICY
    us_filter
ON
    dataset1.table1
GRANT TO
    ('allAuthenticatedUsers')
FILTER USING
    (Region="US");
```

Combine two row access policies for the same grantee into one:

```
CREATE ROW ACCESS POLICY
    shoes_and_blue_products
ON
    dataset1.table1
GRANT TO
    ('user:jon@example.com')
FILTER USING
    (product_category = "shoes" OR color =
    "blue");
```

Row-level security versus other methods

	Security	Recommended for
Authorized views	Vulnerable to carefully crafted queries, query duration, and other types of side-channel attack	When flexibility and performance are most important; for example, when everyone within the same work group shares data.
Row-level access policies		When it is convenient to have all users query the same table; for example, when everyone shares the same dashboard, but some users have access to more data. To provide additional security over views; for example, sharing table slices within your organization.
Separate tables	Complete isolation	When isolation is paramount; for example, when the total number of rows must be secret. Example: Sharing data outside your organization, such as with third-party partners and vendors.

Row-level security limitations

- A table can have up to 100 row access policies.
- Row access policies are not compatible with legacy SQL.
- Row-level security does not participate in query pruning, which is a feature of partitioned tables.
- You cannot apply row-level access policies on JSON columns.
- Creating, replacing, or deleting row-level access policies must be performed with DDL statements.

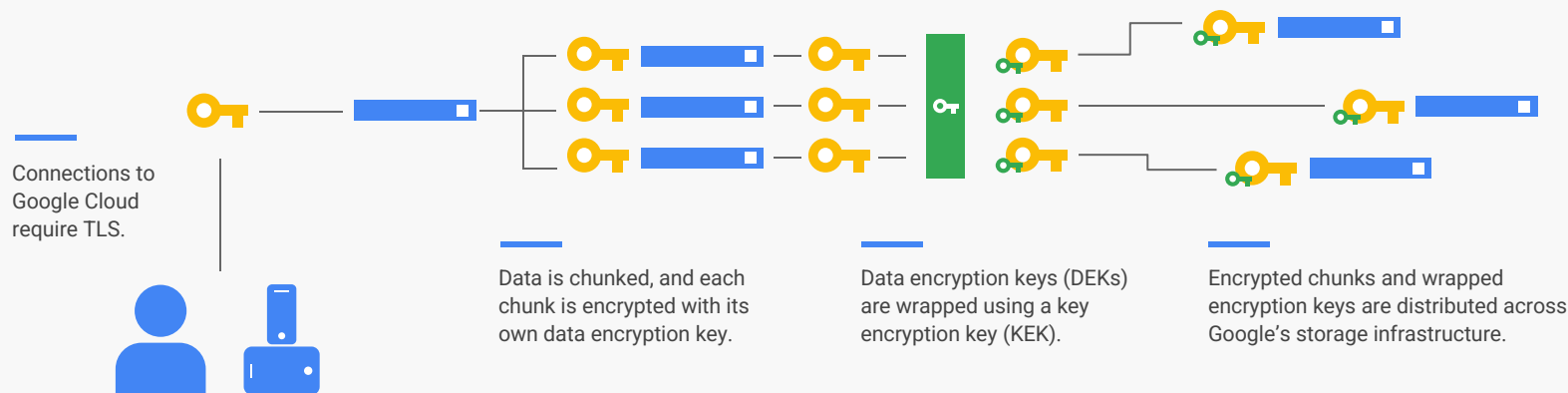
Best practices for row-level security

- Avoid making row access policies that filter on clustered and partitioned columns.
- Restrict user permissions to limit side-channel attacks
- Use row-level security only within organizations, not across organizations.
- Use the ***Filtered Data Viewer*** role with caution.



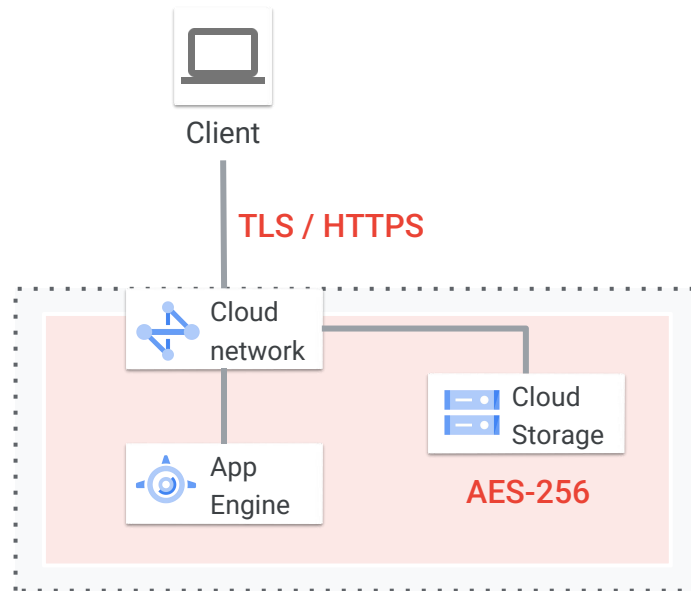
Encryption

Encryption by default in transit and at rest



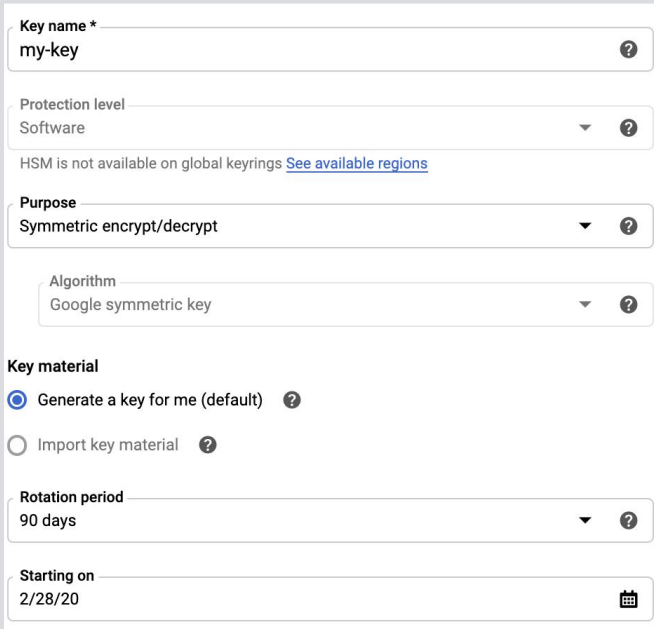
Google Cloud provides server-side encryption of data at rest by default

- Data Encryption Key (DEK) uses AES-256 symmetric key.
- Keys are encrypted by Key Encryption Key (KEK).
- Google controls root keys in Cloud Key Management Service (KMS).
- Keys are automatically periodically rotated.
- On-the-fly decryption happens on authorized user access with no visible performance impact.



For compliance reasons, you may need to manage your own keys

- Customer-managed encryption keys are created in the cloud using Cloud KMS.
- You create the keys and specify the rotation frequency.
- You can then select your keys when creating storage resources like bucket and disks.



The image shows a screenshot of the Google Cloud KMS 'Create key' form. The form is divided into several sections with labels and dropdown menus. The 'Key name' field is set to 'my-key'. The 'Protection level' is set to 'Software', with a note below stating 'HSM is not available on global keyrings' and a link to 'See available regions'. The 'Purpose' is set to 'Symmetric encrypt/decrypt'. The 'Algorithm' is set to 'Google symmetric key'. Under 'Key material', the 'Generate a key for me (default)' option is selected. The 'Rotation period' is set to '90 days'. The 'Starting on' date is set to '2/28/20'.

Key name *

my-key

Protection level

Software

HSM is not available on global keyrings [See available regions](#)

Purpose

Symmetric encrypt/decrypt

Algorithm

Google symmetric key

Key material

☒ Generate a key for me (default)

☐ Import key material

Rotation period

90 days

Starting on

2/28/20

AEAD encryption functions

Allows users to build crypto-shredding style solutions by leveraging encrypt/decrypt functions in BigQuery SQL.

Alternative to doing expensive DML delete curation on legacy data by managing decryption keys and leveraging JOINS.

Data table
(encrypted fields)

Data table
(encrypted fields)

Data table
(encrypted fields)

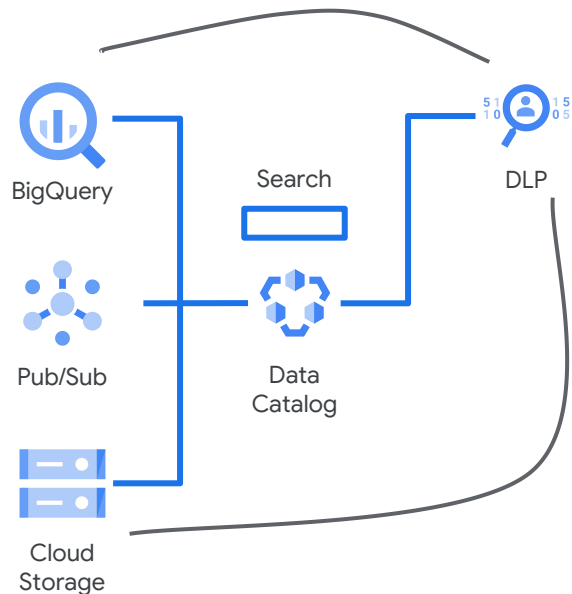
Key material table
(one key per ID)

A blue square containing the white text '05'. To the right of the square is a yellow triangle pointing downwards.

05

Data Discovery and Governance

Use Data Catalog and Cloud Data Loss Prevention API for guarding PII



Data Catalog

Simplify data discovery at any scale:

Fully managed metadata management service with no infrastructure to set up or manage

Unified view of all datasets:

Central and secure data catalog across Google Cloud with metadata capture and tagging

Data governance foundation:

Security compliance with access level controls along with Cloud Data Loss Prevention integration for handling sensitive data

The Cloud Data Loss Prevention API can be used to protect sensitive data by finding it and redacting it

- Scans data in Cloud Storage, BigQuery, or Firestore.
- Can also scan images.
- Detects many different types of sensitive data, including:
 - Emails
 - Credit cards
 - Tax IDs
- You can add your own information types.
- Can delete, mask, tokenize, or just identify the location of the sensitive data.

CREDIT_CARD_NUMBER ✕

EMAIL_ADDRESS ✕

GCP_CREDENTIALS ✕

IMEI_HARDWARE_ID ✕

IP_PASSPORT ✕

MAC_ADDRESS ✕

PASSPORT ✕

MAC_ADDRESS_LOCAL ✕

PHONE_NUMBER ✕

US_BANK_ROUTING_MICR ✕

US_EMPLOYER_IDENTIFICATION_NUMBER ✕

US_INDIVIDUAL_TAXPAYER_IDENTIFICATION_NUMBER ✕

US_SOCIAL_SECURITY_NUMBER ✕

US_VEHICLE_IDENTIFICATION_NUMBER ✕

Autotagging PII data in BigQuery with DLP integration



column:credit_card_number: DLP Result Scan Template (travel-bookings-236421.dlp_result_scan_template)

Attribute	Display name	Value
scan_date_time	Date Time of Scan	04/04/2019 04:44
result	Result	CREDIT_CARD_NUMBER
has_pii	Has PII Information	true
num_rows_scan	Number of Rows to Scan in DLP	1000
result_percentual	Result percentual	1
all_result	all result	CREDIT_CARD_NUMBER(LIKELY : 100.0%);
num_rows_table	Number of Rows in the table scanned	13000010
result_likelihood	Results likelihood	LIKELY
num_match	Number of Results	1000

Auto-tagging
of PII data
using DLP

Structured tags to enrich metadata



Tag templates

Define *template* for creating tags

Attributes

ID	Display Name	Type
approved_by_data_governance	All approved data assets need to be marked with this seal of approval. Data assets without this seal of approval will be deleted after 30 days. This is needed to enforce compliance	BOOL
data_classification	Data Classification	ENUM
data_lifecycle	Data Lifecycle	ENUM
delete_by_date	This data asset needs to be deleted by the specified date	STRING
has_pii	Has PII	BOOL
retain_till_date	This data asset needs to be retained till the specified date	STRING

Values ▾

- Public
- Sensitive
- Confidential
- Regulatory



Attach *tags* on individual data assets using a template

Data Governance Template (hr-240999108536-2.data_governance_template)

Attribute	Display name	Value
approved_by_data_governance	All approved data assets need to be marked with this seal of approval - Data assets without this seal of approval will be deleted after 30 days - This is needed to enforce compliance	true
data_classification	Data Classification	Sensitive
data_lifecycle	Data Lifecycle	In_prod
delete_by_date	This data asset needs to be deleted by the specified date	10/21/2020
has_pii	Has PII	false
retain_till_date	This data asset needs to be retained till the specified date	11/21/2019

ACL controls on metadata

Control who can **view** the cataloged assets and their **metadata**



Data Catalog viewer

- Discover and read metadata

Control who can **enrich** the metadata with **tags** and **view** the tags



Tag template owner, creator

- Create tag templates, use and view tags of that template

Tag template user

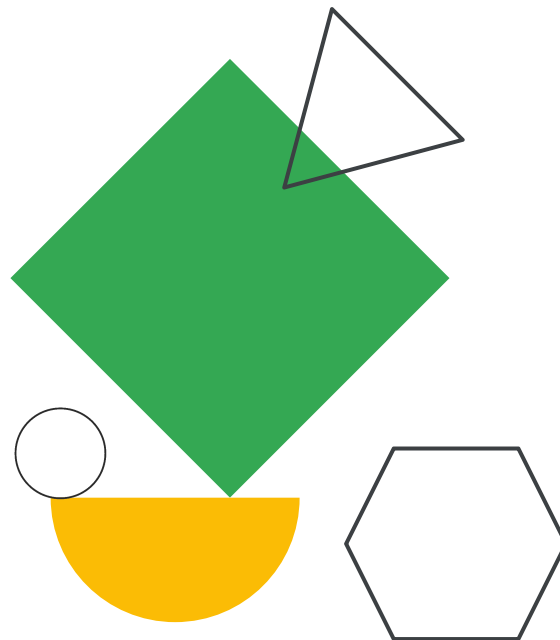
- Use and view tags of a template

Tag template viewer

- View tags of a template

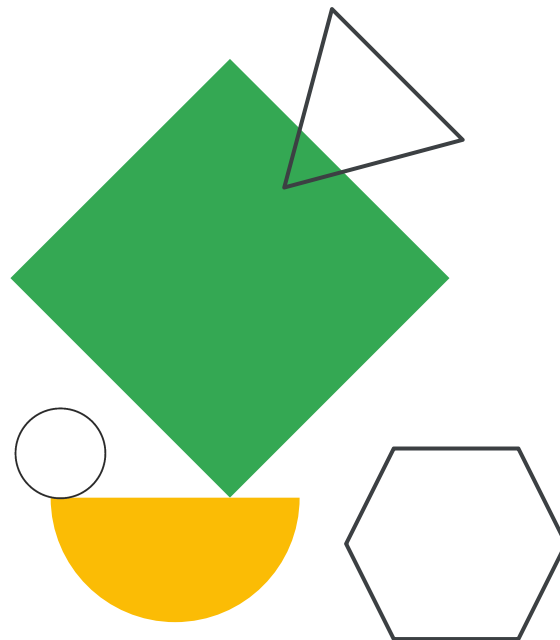
Demo

Using Data Catalog to explore
metadata of BigQuery tables



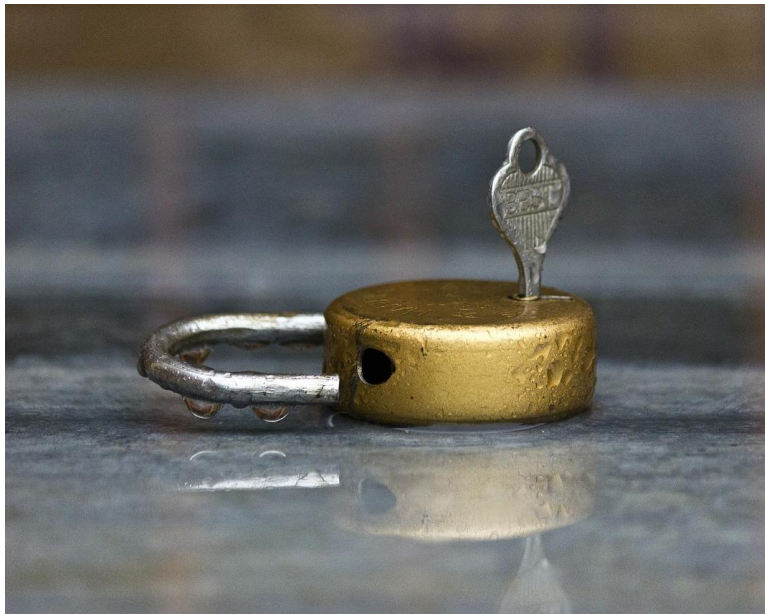
Demo

Finding PII in your dataset with
the DLP API

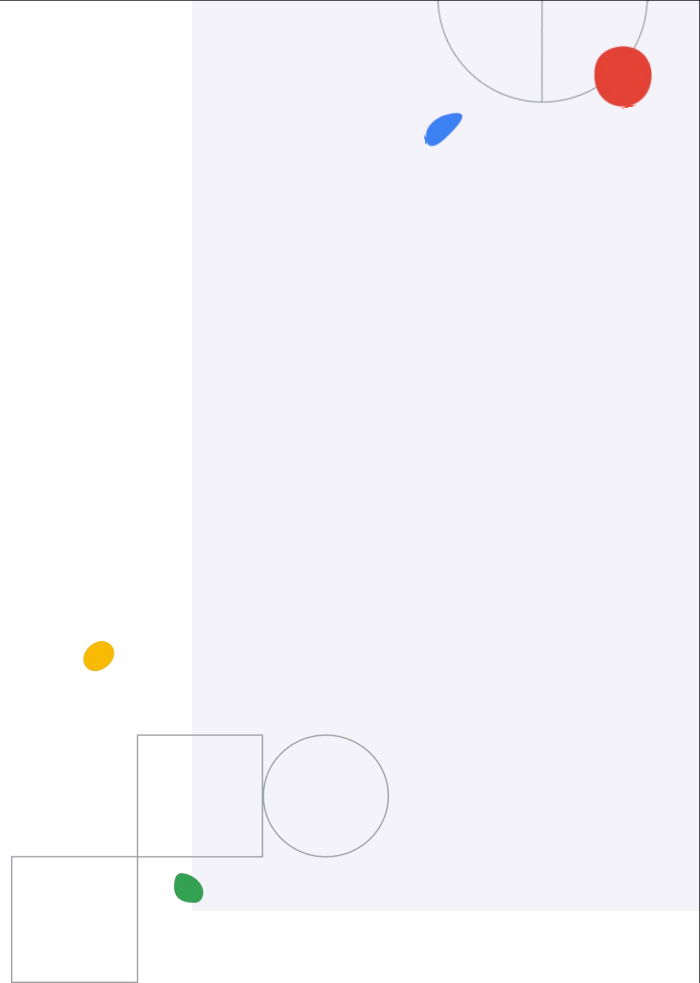


Avoid access pitfalls

- Dataset users should have the minimum permissions needed for their role.
- Use separate projects or datasets for different environments (e.g., DEV, QA, PRD).
- Audit roles periodically.



Questions?



Lab (60 min)

Creating a BigQuery Authorized View

1:00:00